Programming Assignment: Model-Based Testing — State-Machine-Based Test Generation, Automation, and Evaluation

Case Study: Smart Parcel Locker Management System
Primary Focus: Executable state modelling, automated test generation, test execution, mutation testing, and experimental evaluation.

1. Assignment Overview

The objective of this assignment is to design, implement, and evaluate an automated Model-Based Testing (MBT) framework for a Smart Parcel Locker Management System.

In conventional software testing, test cases are often written manually based on requirements or user scenarios. In Model-Based Testing, an abstract representation of the system's behaviour is developed first. This model is then used to automatically generate test sequences, execute them against the system under test, and evaluate whether the implementation behaves as expected.

In this assignment, you are required to go beyond writing conventional test cases. You will develop an executable state-machine model, implement multiple test-generation strategies, automate test execution, introduce faults into the system, and conduct experiments to evaluate the effectiveness of your testing approach.

The assignment is designed to assess your ability to apply software testing concepts through actual programming and experimentation. A report containing only theoretical explanations, diagrams, or manually written test cases will not be considered sufficient.

2. Learning Objectives

By completing this assignment, you should be able to:

1. Analyse system requirements and identify states, events, transitions, guards, and invariants.
2. Develop an executable state-machine model of a software system.
3. Design and implement automated test-generation strategies.
4. Generate valid and invalid test sequences from a behavioural model.
5. Develop an automated test execution framework and test oracle.
6. Apply mutation testing and fault injection to assess test effectiveness.
7. Measure state, transition, and transition-pair coverage.

8. Design controlled experiments to compare different test-generation strategies.
9. Analyse experimental results and identify limitations of different testing approaches.

3. Case Study: Smart Parcel Locker Management System

Consider a parcel delivery company operating a network of smart lockers. Each locker consists of multiple compartments that customers and delivery agents can use to deposit and retrieve parcels.

A customer may reserve a compartment, a delivery agent may deposit a parcel, and the customer may subsequently retrieve it using an authentication code. A compartment may also become unavailable due to maintenance.

The system must enforce appropriate operational rules. For example:

- A parcel cannot be deposited in an unavailable compartment.
- A customer cannot retrieve a parcel without valid authentication.
- An already retrieved parcel cannot be retrieved again.
- A reservation cannot be cancelled if the applicable cancellation rules do not permit it.
- An expired reservation must be handled according to the defined expiry policy.

For this assignment, you will initially model one locker compartment independently. You may extend your implementation to support multiple compartments, but the core model and evaluation must remain clear and reproducible.

3.1 Required States

Your state-machine model must include, at a minimum, the following conceptual states:

State	Description
AVAILABLE	The compartment is empty and available for reservation.
RESERVED	The compartment has been reserved, but the parcel has not yet been deposited.
OCCUPIED	A parcel has been deposited and is awaiting collection.
MAINTENANCE	The compartment is temporarily unavailable due to maintenance.

You may introduce additional states or state variables where necessary. For example, you may maintain:

- Reservation ID
- Parcel ID
- Access or authentication code
- Reservation timestamp
- Reservation expiry timestamp
- Authentication status
- Maintenance status

Any additional state variables must be documented and their role in the model must be clearly explained.

3.2 Required Operations and Events

Your model must support the following operations:

Operation	**Description**
RESERVE	Reserve an available compartment.
DEPOSIT	Deposit a parcel into a valid reservation.
RETRIEVE	Retrieve a deposited parcel after successful authentication.
EXPIRE	Expire an active reservation when its expiry condition is satisfied.
CANCEL	Cancel a reservation when cancellation is permitted.
DISABLE	Make an available compartment unavailable for maintenance.
ENABLE	Return a compartment from maintenance to the available state.
INVALID_ACCESS	Attempt an operation using invalid authentication credentials.

For each operation, you must define:

1. Preconditions.
2. Guard conditions.
3. Valid and invalid input conditions.
4. State changes.

5. Expected operation results.
6. Behaviour when an operation is rejected.

For example, consider the following questions:

- What should happen if a customer attempts to reserve an occupied compartment?
- Can a parcel be deposited using an incorrect access code?
- Can a parcel be retrieved before it has been deposited?
- What happens if a reservation expires before the parcel is deposited?
- Can a compartment enter maintenance while it contains a parcel?
- What should happen if an invalid operation is attempted?

3.3 Expiration and Maintenance Semantics

The EXPIRE operation represents the expiration of an active reservation. It is accepted only when the locker is in the RESERVED state, and the expiry condition is satisfied.

Upon successful expiration, the locker immediately transitions from RESERVED to AVAILABLE, and all associated customer and reservation information is cleared.

The EXPIRE operation must be rejected if the locker is not in the RESERVED state or if the expiry condition has not been satisfied.

For maintenance management, DISABLE transitions an AVAILABLE locker to MAINTENANCE, while ENABLE transitions a MAINTENANCE locker back to AVAILABLE.

For the supplied reference implementation, the expiry condition is represented through an explicit Boolean input (expired=True or expired=False). Students should not rely on actual system time or wall-clock calculations.

The assignment intentionally leaves some policy decisions open. You are expected to make reasonable assumptions, document them, and apply them consistently in your model and implementation.

4. Required Assignment Tasks

Task 1: Requirements Analysis and Executable State Model

Develop a behavioural model of the Smart Parcel Locker Management System.

Your work must include:

- Identification of system states.
- Identification of events and operations.
- Definition of transition conditions and guards.
- Definition of state updates.
- Definition of valid and invalid operation outcomes.
- Identification of important state invariants.

You must implement the state-machine model in executable code.

A state-transition diagram may be included in your report for explanation, but a diagram alone will not fulfil this requirement.

Your model should be able to:

- Identify the current state of the system.
- Determine whether an operation is valid in the current state.
- Execute valid transitions.
- Reject invalid transitions.
- Return an expected outcome for each operation.
- Maintain the consistency of the system state.

Important: Keep the executable model logically separate from the test-generation algorithms. This will make it easier to compare different testing strategies.

Task 2: Implement at Least Three Test-Generation Strategies

You are required to implement at least three meaningfully different test-generation strategies using your executable state model.

The purpose is to investigate how different strategies explore the system's behaviour and how effectively they detect faults.

The following strategies are suggested.

Strategy A: Random or Model-Guided Random Testing

Develop a generator that produces test sequences by randomly selecting operations or transitions.

You may use a purely random approach or a model-guided approach that uses the current state to influence operation selection.

Your implementation should explain:

- How operations are selected.
- How valid and invalid operations are handled.
- How sequence length is controlled.
- How random seeds are used to reproduce experiments.

Strategy B: Coverage-Directed Testing

Develop a strategy that attempts to maximize coverage of the system's states and transitions.

For example, the generator may prioritize:

- States that have not yet been visited.
- Transitions that have not yet been executed.
- Less frequently exercised parts of the model.

The objective is to generate test sequences that systematically explore the model rather than relying entirely on random selection.

Strategy C: Transition-Oriented or Path-Based Testing

Implement a strategy that considers sequences of transitions rather than individual operations.

For example, you may use:

- Transition-pair coverage.
- Bounded path exploration.
- Systematic exploration of selected state-transition paths.
- Another justified sequence-based strategy.

The objective is to investigate whether considering transition sequences can reveal faults that individual transition coverage may miss.

Important requirements:

- All three strategies must be genuinely different in their selection or exploration mechanisms.
- Simply changing a random seed or a configuration parameter does not count as implementing a new strategy.
- Document the algorithm used by each strategy.
- Explain the advantages and limitations of each strategy.
- Make each strategy independently executable.

You may propose alternative strategies, provided that they are technically justified and approved by the instructor where necessary.

Task 3: Automated Test Execution and Test Oracle

Develop an automated test execution framework.

The framework must take generated test sequences and execute them against the provided reference implementation or system under test (SUT).

Your framework must compare the actual behaviour of the SUT with the expected behaviour derived from your model.

For every test execution, record:

Information	Description
Test ID	Unique identifier of the generated test.
Strategy	Strategy used to generate the test.
Sequence	Complete sequence of operations.
Initial state	State before test execution.
Expected result	Result predicted by the model.
Actual result	Result returned by the SUT.
Final state	Expected and actual final states.
Test outcome	Pass or Fail.
Failure details	Explanation of any discrepancy.

Your test oracle must check more than just the return value of an operation. It should also verify relevant state invariants.

For example:

- A retrieved parcel should not remain available for retrieval.
- An unauthorized retrieval should not change the compartment state.
- An expired reservation must be cleared according to the defined expiry policy, and the compartment must return to the AVAILABLE state.
- A rejected operation should not cause unintended state changes.

You should also consider how to avoid making your oracle simply a duplicate of the SUT implementation. Explain how you have designed the expected-behaviour logic to reduce the possibility of common implementation errors.

Task 4: Fault Injection and Mutation Testing

One of the most important objectives of this assignment is to evaluate whether your generated tests can actually detect faults.

For this purpose, you are required to evaluate your testing framework against at least eight faulty versions (mutants) of the system.

Each mutant should represent a distinct and plausible implementation defect.

Examples of possible fault categories include:

- Allowing reservation of an already occupied compartment.
- Allowing deposits during maintenance.
- Allowing retrieval without authentication.
- Incorrectly expiring a reservation before its expiry condition is satisfied.
- Failing to clear the compartment state after retrieval.
- Retaining stale authentication credentials after retrieval.
- Allowing cancellation when cancellation should not be permitted.
- Changing system state after an invalid deposit attempt.

These examples illustrate possible defect categories. Your report must describe the actual mutants used in your experiments.

For each mutant, determine:

1. What defect has been introduced?
2. Which test-generation strategy detects the defect?
3. Which generated test sequence exposes it?
4. What incorrect behaviour is observed?
5. Does the mutant survive your testing?
6. If it survives, why was it not detected?

A mutant that survives testing should be investigated. It may be equivalent under the defined model, unreachable, or simply not exercised by your test-generation strategies.

You must explain the reason rather than merely reporting the final mutation score.

The instructor may provide or activate faulty implementations separately. Any instructor-supplied confidential material must not be distributed or modified without permission.

Task 5: Experimental Evaluation and Comparison

Conduct a controlled experiment to compare the effectiveness of your three test-generation strategies.

The objective is to determine how well each strategy explores the model and detects injected faults.

Use the following minimum experimental protocol:

Parameter	**Minimum Requirement**
Number of strategies	3
Test sequences per strategy	500
Maximum sequence length	20 operations
Random seeds	5 for stochastic strategies
Number of mutants	At least 8
State coverage	Must be measured
Transition coverage	Must be measured

Transition-pair coverage	Report where applicable
Mutation score	Must be calculated
Execution time	Must be reported

You may use a different experimental configuration if you provide a clear technical justification.

Required Evaluation Metrics

1. State Coverage

Measure the proportion of model states visited during test execution.

$$State\ Coverage = \frac{\text{Number of states visited}}{\text{Total number of model states}} \times 100$$

2. Transition Coverage

Measure the proportion of model transitions exercised by the generated tests.

$$Transition\ Coverage = \frac{\text{Number of transitions exercised}}{\text{Total number of model transitions}} \times 100$$

3. Transition-Pair Coverage

Measure the proportion of possible defined adjacent transition pairs exercised by the generated test sequences.

You must clearly explain how transition pairs are identified and counted in your implementation.

4. Mutation Score

Measure the percentage of non-equivalent mutants detected by the generated tests.

$$Mutation\ Score = \frac{\text{Number of non-equivalent mutants detected}}{\text{Total non-equivalent mutants considered}} \times 100$$

If a mutant is potentially equivalent or cannot be evaluated conclusively, identify it separately.

5. Execution Time

Measure the time required by each strategy to generate and execute its test sequences.

Where possible, distinguish test-generation time from test-execution time.

Experimental Results

Present your results using suitable tables and charts.

Your discussion should address questions such as:

- Which strategy achieved the highest state coverage?
- Which strategy achieved the highest transition coverage?
- Which strategy detected the most mutants?
- Did higher coverage necessarily result in a higher mutation score?
- Which strategy generated tests more efficiently?
- Did any strategy repeatedly generate redundant test sequences?
- What are the strengths and limitations of each approach?

Your conclusions must be based on your experimental results rather than assumptions.

5. Implementation Requirements

The following implementation requirements apply:

1. Use Python 3, or any other programming language; however, Python is recommended.
2. Organize your project into logical modules, such as:
    - State model
    - Test generators
    - Test execution engine
    - Test oracle
    - Coverage metrics
    - Mutation testing

    - Experiment runner
3. Provide a README file explaining installation, dependencies, and execution instructions.
4. Use random seeds to make stochastic experiments reproducible.
5. Store experimental results in a machine-readable format such as CSV or JSON.
6. Handle invalid operations and execution errors appropriately.
7. Ensure that the project can be executed in a clean environment using your documented instructions.
8. Do not submit only screenshots, manually prepared result tables, or pseudocode.

The implementation should demonstrate sound software engineering practices, including modularity, readability, meaningful naming, and appropriate separation of concerns.

6. Student Starter Package and Provided Resources

A Student Starter Package ZIP file will be provided along with this assignment. It contains the initial project structure and supporting files to help you get started with the implementation.

The package is intended to provide a common starting point for all students. It does not contain a complete solution to the assignment.

Please note that in the Student Starter Package ZIP file, the supplied reference SUT implements the expiration of a reservation as a direct transition from RESERVED to AVAILABLE. The implementation also clears the associated reservation information upon successful expiration. You are required to develop your independent executable model based on the behavioural requirements specified in Section 3. The supplied reference SUT should be treated as the implementation under test, not as the source of expected behaviour.

6.1 Contents of the Student Starter Package

The ZIP package includes:

- Reference System Under Test (SUT): An initial implementation of the Smart Parcel Locker Management System that you will use as the system under test.
- Type Definitions: Initial data structures and definitions required by the project.

- Starter Model Module: A placeholder for implementing your executable state-machine model.
- Test Generator Modules: Initial module structures for implementing your three test-generation strategies.
- Test Runner Module: A starting structure for implementing automated test execution.
- Metrics Module: A placeholder for implementing coverage and other evaluation metrics.
- Smoke Tests: Basic tests to help verify that the starter environment is configured correctly.
- README and Requirements: Initial setup instructions and dependency information.

6.2 How to Use the Starter Package

You are expected to:

1. Download the Student Starter Package ZIP provided with this assignment.
2. Extract it into your working directory.
3. Read the README file and follow the environment setup instructions.
4. Examine the reference SUT and understand its operations and expected behaviour.
5. Identify the incomplete modules and implement the required functionality.
6. Extend or modify the provided project structure where necessary to fulfil all assignment requirements.
7. Implement your state model, test-generation strategies, execution framework, oracle, and evaluation metrics.
8. Add your own test cases and supporting scripts.

6.3 Important Instructions

- The starter package is provided to establish a common project structure; it is not a complete implementation.
- You are responsible for completing the TODO sections and implementing all required functionality.

- You may reorganize or extend the provided code, provided that the required functionality remains available and your implementation is properly documented.
- You must not assume that passing the initial smoke tests means the assignment is complete.
- Your final implementation must satisfy all requirements specified in this assignment.
- You must be able to explain the purpose and implementation of every important module during the individual viva.

7. Expected Deliverables

You are required to submit the following:

1. Executable Source Code

The complete implementation, including:

- State-machine model.
- Three or more test-generation strategies.
- Automated test execution engine.
- Test oracle and invariant checks.
- Coverage measurement.
- Mutation testing support.
- Experimental execution scripts.

2. Technical Report

The report should explain:

- System requirements and assumptions.
- State-machine design.
- Implementation architecture.
- Test-generation algorithms.
- Test execution and oracle design.
- Mutation-testing approach.

- Experimental setup and methodology.
- Results and comparative analysis.
- Limitations and threats to validity.

3. Experimental Results

Include:

- Raw test execution results.
- Coverage measurements.
- Mutation-testing results.
- Execution-time measurements.
- Relevant tables and charts.

4. Demonstration

Provide a short demonstration showing:

- Execution of the state-machine model.
- Generation of test sequences.
- Execution of generated tests.
- Detection of at least one injected fault.
- Presentation of the experimental results.

5. README File

The README must contain clear instructions for:

- Setting up the environment.
- Installing dependencies.
- Running the application.
- Executing each test-generation strategy.
- Running mutation experiments.
- Reproducing the reported results.

Student submission: Submit your completed project as a ZIP file containing the implemented source code, README, experiment scripts, raw results, and technical report.

8. Suggested Technical Report Structure

You may follow the structure below:

1. Introduction: Assignment objectives and overview of the case study.
2. Requirements Analysis: Functional requirements, assumptions, and behavioural rules.
3. State-Machine Model: States, variables, events, guards, transitions, and invariants.
4. Test-Generation Strategies: Design and implementation of each strategy.
5. Test Execution Framework: Runner architecture and oracle implementation.
6. Mutation Testing: Description of mutants and fault-detection methodology.
7. Experimental Setup: Environment, seeds, sequence limits, metrics, and configurations.
8. Results: Coverage, mutation scores, execution time, and other measurements.
9. Discussion: Comparative analysis, observations, limitations, and threats to validity.
10. Conclusion: Key findings and possible improvements.

9. Assessment Criteria

The assignment will be evaluated according to the following rubric:

Assessment Component	Marks
Requirements analysis, assumptions, and testing strategy	10
Executable state model and transition semantics	15
Three or more test-generation strategies	20
Automated runner, oracle, and invariant checks	15
Mutation/fault-injection testing	15
Experimental evaluation and interpretation	15
Code quality, reproducibility, report, and demonstration	10
Total	100

10. Individual Viva and Unseen Change Request

Each student must be able to explain and modify their implementation individually.

During the viva, the instructor may introduce a new requirement or change an existing system rule. For example:

- Maintenance may now be initiated while a compartment is occupied.
- A reservation may be cancelled under an additional condition.
- The parcel retrieval authentication mechanism may change.
- The parcel expiry policy may be modified.
- A new state or operation may be introduced.

You may be asked to:

1. Identify the impact of the change on the state-machine model.
2. Modify the executable model accordingly.
3. Explain which test-generation strategies are affected.
4. Update the test oracle and relevant test cases.
5. Execute the updated tests.
6. Explain how the change affects coverage and fault detection.

The viva is intended to assess your actual understanding of the submitted work. You should therefore be familiar with the complete implementation, including the algorithms, design decisions, and experimental results.

11. Academic Integrity and Use of AI Tools

This assignment is intended to assess your ability to design and implement testing solutions independently.

You can use AI-assisted tools for coding. Any permitted use of AI tools must be disclosed as required.

You are responsible for understanding, validating, and being able to explain all submitted code. Submitting code that you cannot explain, debug, or modify during the viva may affect your assessment.

The use of automated coding tools does not replace the requirement to understand the underlying testing concepts, algorithms, and implementation decisions.

12. Final Submission Checklist

Before submitting, ensure that:

- You have implemented an executable state-machine model.
- You have documented all system assumptions and transition rules.
- You have implemented at least three genuinely different test-generation strategies.
- You have developed an automated test runner and oracle.
- You have implemented state and transition coverage measurement.
- You have evaluated at least eight faulty system variants.
- You have calculated and discussed mutation scores.
- You have conducted the required experiments.
- You have included raw experimental results.
- You have provided a complete technical report.
- You have included setup and execution instructions.
- You can explain and modify your implementation during the individual viva.

The central objective of this assignment is not simply to generate a large number of test cases. It is to investigate how different model-based testing strategies explore system behaviour, detect faults, and provide measurable evidence of software quality. Your implementation and experimental findings should demonstrate this understanding.

Good luck!